

Module 1

Python and Verification Basics

Learning objectives

- Understand Python for verification and verification fundamentals

Prerequisites

- See module README

Learning path

Learning Path

Diagram: Learning Path

(render with mmdc when available)

flowchart LR

T1["Python Classes and Inheritance"]

T2["Decorators and Context Managers"]

T1 --> T2

T3["Async/Await for Simulation"]

T2 --> T3

Understand Python for verification and verification fundamentals

Overview

- This module establishes the foundation for verification work. You'll learn essential Python concept...

How to learn this module

Suggested learning path

- Complete Module 0 (``./scripts/module0.sh --verify-only``) so Verilator, cocotb, and pyuvm are available...
- Skim ``docs/MODULE1.md`` and run pure Python examples first — no simulator required for OOP/async fou...
- Review ``module1/dut/simple_gates/`` RTL (AND gate, counter) before opening cocotb tests
- Understand the testbench mental model: DUT ← driver/stimulus ← test ← checker ← monitor
- Keep venv active; cocotb Makefiles expect ``SIM=verilator`` and ``COCOTB_REDIRECT_OUTPUT=1`` defaults f...

Design architecture

1. RTL block architecture

- DUTs in
 - ``module1/dut/simple_gates/`` —
 - ``and_gate.v`` (combinational) and
 - ``counter.v`` (enable, active-...
- Synchronous blocks: clock, active-low reset, multi-bit data paths on the counter
- Flat hierarchy — no buses; ideal for first cocotb signal access and pyuvvm hook-up

Rtl Architecture

```
Diagram: Rtl Architecture
(render with mmdc when available)
flowchart TB
  subgraph dut["module1/dut"]
    AND[simple_gates/and_gate]
    CTR[counters/counter]
  end
  CLK[clk / rst_n] --> AND
```

2. Python verification layer architecture

- ``module1/examples/`` — pure Python (transactions, decorators, async, data structures, logging)
- ``module1/tests/cocotb_tests/`` — coroutine testbenches driving RTL via cocotb handles
- ``module1/tests/pyuvvm_tests/`` — minimal UVM-style hierarchy introduction
- Transaction classes in ``examples/python_basics/transaction.py`` model stimulus items reused in TB th...

Verification Architecture

```
Diagram: Verification Architecture
(render with mmdc when available)
flowchart LR
    subgraph py["Python layer"]
        EX[examples: transactions]
        CT[cocotb_tests]
        PT[pyuvvm_tests]
    end
    end
```

3. Execution pipeline

- Build: Verilator compiles
`module1/dut/simple\_gates/\*.v` via cocotb Makefile
(`VERILOG\_SOURCES`, to...
- Sim: cocotb loads Python test module; coroutines
advance time on ` RisingEdge(clk)` and combinationa...
- Check: assertions in test functions plus cocotb
`TestSuccess` / `TestFailure`; pyuvvm tests report UVM...
- Orchestrator path: `module1.sh --cocotb-tests` wraps
`make` with venv and simulator checks

4. Testbench mental model

- DUT (RTL) ← driver/stimulus ← test ←
checker/scoreboard ← monitor → observations
- Simulation time advances on clock edges; Python coroutines (``async`/`await``) align with cocotb
- Assertions and logging close the loop between expected and actual behavior
- Python examples teach patterns (transactions, async) reused when pyuvvm sequences appear in Module 3

DUT — AND gate ports

```
module and_gate (  
    input  wire a,  
    input  wire b,  
    output reg  y  
);  
  
    always @(*) begin  
        y = a & b;  
    end  
  
endmodule
```

cocotb — drive inputs and assert

```
@cocotb.test()
async def test_and_gate_basic(dut):
    """
    Basic AND gate test.

    Tests all input combinations.
    """
    # Initialize inputs
    dut.a.value = 0
    dut.b.value = 0

    # Wait for initial values to settle
    await Timer(10, units="ns")

    # Test case 1: 0 & 0 = 0
    dut.a.value = 0

    # ...
```

Key files to study

Open these in the repo

- ``module1/dut/simple_gates/and_gate.v`` — 2-input AND; first combinational DUT for cocotb handles
- ``module1/dut/simple_gates/counter.v`` — clocked counter with enable/reset for sequential labs
- ``module1/examples/python_basics/transaction.py`` — transaction class with ``__eq__``/``__hash__`` patter...
- ``module1/tests/cocotb_tests/test_and_gate.py`` — directed cocotb vectors and edge triggers
- ``module1/tests/pyuvm_tests/test_and_gate_uvm.py`` — minimal pyuvm hierarchy on the AND gate
- ``scripts/module1.sh`` — routes Python examples and ``--cocotb-tests`` / ``--pyuvm-tests`` regressions

Verification & testing methods

1. cocotb testing methods

- Tests in ``test_and_gate.py``,
``test_counter.py``: reset,
directed vectors, edge cases
- Run: ``cd``
`module1/tests/cocotb_tests &&`
`make SIM=verilator`
`TEST=test_and_gate``
- Triggers: `` RisingEdge(clk)``,
``ReadOnly` / `Write`` phases for
signal stability

Testing Methods

Diagram: Testing Methods
(render with mmdc when available)
flowchart TD
S[Stimulus: directed vectors] --> DUT[DUT]
DUT --> M[Monitor / read signals]
M --> C{Match expected?}
C -->|yes| P[PASS]
C -->|no| F[FAIL + log]

2. pyuvm and Python unit-style checks

- ``./scripts/module1.sh --pyuvm-tests`` — compile/run introductory pyuvm tests against simple DUTs
- Examples run without simulator (``python3 module1/examples/...``) to teach OOP patterns first
- Error-handling example shows structured logging for debuggable regressions

3. Step-by-step lab execution

- 1. Run Python track: `./scripts/module1.sh`` (or individual `--python-basics``, `--async-await``, ...)
- 2. cocotb AND gate: `cd module1/tests/cocotb_tests && make SIM=verilator TEST=test_and_gate``
- 3. cocotb counter: `make SIM=verilator TEST=test_counter`` — observe reset and enable sequencing
- 4. pyuvm entry: `./scripts/module1.sh --pyuvm-tests`` — confirm UVM-style test completes without fat...
- 5. Full regression: `./scripts/module1.sh --all-tests`` before MODULE1 assessment sign-off

4. Closure and self-check

- ``./scripts/module1.sh --all-tests`` runs required examples and simulator-backed tests
- Combine directed cocotb tests with transaction equality (``__eq__``, ``__hash__``) from Python examples
- Assessment: OOP for verification, basic TB structure, cocotb + pyuvvm entry points

Makefile — build and run flow

```
# Makefile for Module 1 cocotb tests
# Usage: make SIM=verilator test_and_gate

# Default simulator
SIM ?= verilator

# Python files
PYTHON_FILES = test_and_gate.py test_counter.py

# Verilog files
VERILOG_FILES = ../../dut/simple_gates/and_gate.v
               ../../dut/simple_gates/counter.v

# Test to run
TEST ?= test_and_gate
```

```
# Cocotb variables
```

Syllabus topics

1. Python Classes and Inheritance for Verification (1/6)

- Object-Oriented Programming Basics
- Classes and objects
- Instance variables and methods
- Class variables and methods
- Encapsulation concepts
- Inheritance in Python

1. Python Classes and Inheritance for Verification

(2/6)

- Single inheritance
- Multiple inheritance (MRO)
- Method overriding
- ``super()`` function usage
- Special Methods (Dunder Methods)
- ``__init__()`` for initialization

1. Python Classes and Inheritance for Verification

(3/6)

- ``__str__()`` and ``__repr__()`` for string representation
- ``__eq__()`` for equality comparison
- ``__hash__()`` for hashable objects
- Class Design Patterns for Verification
- Factory pattern basics
- Singleton pattern

1. Python Classes and Inheritance for Verification (4/6)

- Builder pattern
- Strategy pattern
- Base ``Transaction`` class with class variables
 (``_id_counter``)
- Instance variables (``id``, ``data``, ``timestamp``)
- Special methods: ``__init__()``, ``__str__()``,
 ``__repr__()``, ``__eq__()``, ``__hash__()``
- Inheritance: ``ReadTransaction`` and
 ``WriteTransaction`` inherit from ``Transaction``

1. Python Classes and Inheritance for Verification (5/6)

- Method overriding: Child classes override
 `\_\_str\_\_()` method
- Using `super()` to call parent class methods
- Transaction(id=1, data=4660, timestamp=0)
- ReadTransaction(id=2, address=0x1000, data=0xDEAD)
- WriteTransaction(id=3, address=0x2000, data=0xBEEF)
- Class Variables: `\_id\_counter` is shared across all
 instances

1. Python Classes and Inheritance for Verification (6/6)

- Instance Variables: Each transaction has its own
 `id`, `data`, `timestamp`
- Special Methods: Enable Pythonic behavior (string
 representation, equality, hashing)
- Inheritance: `ReadTransaction` and
 `WriteTransaction` extend base functionality
- Method Overriding: Child classes customize string
 representation

2. Decorators and Context Managers (1/5)

- Python Decorators
- Function decorators
- Class decorators
- Decorator syntax and usage
- Common decorators (`@property`, `@staticmethod`, `@classmethod`)
- Context Managers

2. Decorators and Context Managers (2/5)

- ``with`` statement
- ``__enter__()`` and ``__exit__()`` methods
- Context manager protocol
- ``contextlib`` utilities
- Verification-Specific Decorators
- cocotb decorators (``@cocotb.test()``,
``@cocotb.coroutine``)

2. Decorators and Context Managers (3/5)

- Custom decorators for verification
- Timing decorators
- Function Decorators: `@timing_decorator`` and `@log_calls_decorator`` wrap functions
- Decorator Stacking: Multiple decorators can be applied to the same function
- Context Manager Class: `VerificationContext`` implements `__enter__()`` and `__exit__()``
- Function-based Context Manager: `simulation_phase()`` using `@contextmanager``

2. Decorators and Context Managers (4/5)

- Nested Context Managers: Using multiple context managers together
- Logging Integration: Decorators and context managers use Python's logging module
- Decorators: Functions that modify other functions without changing their code
- ``functools.wraps``: Preserves function metadata when decorating
- Context Managers: Ensure proper resource cleanup using ``with`` statements
- Exception Handling: Context managers can handle exceptions in ``__exit__()``

2. Decorators and Context Managers (5/5)

- Logging: Integration with Python's logging module for verification workflows

3. Async/Await for Simulation (1/6)

- Asynchronous Programming Concepts
- Coroutines vs functions
- Event loop basics
- Concurrency vs parallelism
- Python ``async`` and ``await``
- Defining async functions

3. Async/Await for Simulation (2/6)

- Awaiting coroutines
- Async context managers
- Async iterators
- cocotb Coroutines
- cocotb coroutine concept
- ``await`` for simulation time

3. Async/Await for Simulation (3/6)

- Trigger objects
- Coroutine scheduling
- Common Patterns
- Parallel coroutines
- Sequential execution
- Timeout handling

3. Async/Await for Simulation (4/6)

- Exception handling in async code
- Async Functions: Defining coroutines with ``async def``
- Awaiting Operations: Using ``await`` to wait for async operations
- Clock Generation: Simulating clock signals with async functions
- Parallel Execution: Running multiple coroutines concurrently with ``asyncio.gather()``
- Sequential Execution: Running coroutines one after another

3. Async/Await for Simulation (5/6)

- Timeout Handling: Using ``asyncio.wait_for()`` with timeout
- Exception Handling: Catching exceptions in async code
- Queues: Using ``asyncio.Queue`` for communication between coroutines
- Coroutines: Functions defined with ``async def`` that can be paused and resumed
- Event Loop: ``asyncio.run()`` creates and manages the event loop
- Concurrency: Multiple coroutines can run concurrently (not in parallel)

3. Async/Await for Simulation (6/6)

- ``asyncio.gather()``: Runs multiple coroutines concurrently and waits for all
- ``asyncio.wait_for()``: Adds timeout to coroutine execution
- Async Queues: ``asyncio.Queue`` for safe communication between coroutines
- Simulation Time: In cocotb, ``await Timer()`` advances simulation time

4. Verification Fundamentals (1/3)

- What is Verification?
- Design vs verification
- Verification goals
- Verification metrics
- Verification lifecycle
- Testbench Architecture

4. Verification Fundamentals (2/3)

- Testbench components
- Stimulus generation
- Response checking
- Coverage collection
- Verification Levels
- Unit level verification

4. Verification Fundamentals (3/3)

- Block level verification
- System level verification
- SoC level verification

5. Testbench Architecture Basics (1/3)

- Testbench Structure
- Design Under Test (DUT)
- Testbench components
- Interface definition
- Clock and reset generation
- Stimulus Generation

5. Testbench Architecture Basics (2/3)

- Deterministic stimulus
- Random stimulus
- Constrained random
- Directed tests
- Response Checking
- Self-checking testbenches

5. Testbench Architecture Basics (3/3)

- Reference models
- Scoreboards
- Assertions

6. Simulation Flow (1/3)

- Simulation Phases
 - Initialization
 - Reset phase
 - Test execution
 - Cleanup phase
- Time Management

6. Simulation Flow (2/3)

- Simulation time vs real time
- Time units (ns, ps, etc.)
- Clock cycles
- Timing relationships
- Event-Driven Simulation
- Event scheduling

6. Simulation Flow (3/3)

- Delta cycles
- Signal updates
- Trigger conditions

7. Assertions Introduction (1/3)

- What are Assertions?
- Immediate assertions
- Concurrent assertions
- Assertion types
- Basic Assertions
- Simple property checks

7. Assertions Introduction (2/3)

- Signal value assertions
- Timing assertions
- Assertion Best Practices
- When to use assertions
- Assertion placement
- Assertion messages

7. Assertions Introduction (3/3)

- Assertion coverage

8. Python Testing Frameworks (Overview) (1/3)

- pytest Basics
- Test discovery
- Fixtures
- Parametrization
- Markers
- Unit Testing Concepts

8. Python Testing Frameworks (Overview) (2/3)

- Test organization
- Test isolation
- Mocking and stubbing
- Test coverage
- Integration with Verification
- Using pytest with cocotb

8. Python Testing Frameworks (Overview) (3/3)

- Test organization patterns
- Reporting and logging

9. Data Structures for Verification (1/5)

- Lists and Dictionaries
- Common operations
- List comprehensions
- Dictionary comprehensions
- Nested structures
- Collections Module

9. Data Structures for Verification (2/5)

- ``deque`` for queues
- ``defaultdict`` for default values
- ``Counter`` for counting
- ``namedtuple`` for structured data
- Verification-Specific Structures
- Transaction queues

9. Data Structures for Verification (3/5)

- Scoreboard data structures
- Coverage data structures
- ``deque``: Double-ended queue for FIFO/LIFO operations in ``TransactionQueue``
- ``defaultdict``: Dictionary with default values for scoreboard
- ``Counter``: Counting occurrences for statistics
- ``namedtuple``: Structured data for transactions

9. Data Structures for Verification (4/5)

- List Comprehensions: Creating lists from iterables
- Dictionary Comprehensions: Creating dictionaries from iterables
- Scoreboard Pattern: Comparing expected vs actual results
- Coverage Collection: Tracking coverage bins and hit counts
- ``deque``: Efficient FIFO/LIFO operations, thread-safe for async code
- ``defaultdict``: Automatically creates default values for missing keys

9. Data Structures for Verification (5/5)

- ``Counter``: Specialized dictionary for counting occurrences
- ``namedtuple``: Lightweight alternative to classes for simple data structures
- Comprehensions: Pythonic way to create lists/dicts from iterables
- Scoreboard: Common verification pattern for checking results
- Coverage: Tracking which values/conditions have been tested

10. Error Handling and Logging (1/5)

- Exception Handling
- Try/except blocks
- Exception types
- Custom exceptions
- Exception chaining
- Logging in Python

10. Error Handling and Logging (2/5)

- `logging` module basics
- Log levels
- Log formatting
- Log handlers
- Verification Logging
- UVM reporting (preview)

10. Error Handling and Logging (3/5)

- Testbench logging patterns
- Debug logging
- Log analysis
- Custom Exceptions: ``VerificationError``,
``MismatchError``, ``TimeoutError``
- Exception Handling: Try/except blocks with specific exception types
- Exception Chaining: Using ``raise ... from ...`` to chain exceptions

10. Error Handling and Logging (4/5)

- Retry Logic: Implementing retry mechanisms with error handling
- Logging Levels: DEBUG, INFO, WARNING, ERROR, CRITICAL
- Log Handlers: File and console handlers
- Log Formatting: Custom log message formats
- Verification Statistics: Tracking pass/fail/error counts
- Custom Exceptions: Create domain-specific exceptions for better error handling

10. Error Handling and Logging (5/5)

- Exception Chaining: Preserve original exception context with ``raise ... from ...``
- Retry Logic: Implement robust error recovery mechanisms
- Logging Levels: Use appropriate levels (DEBUG < INFO < WARNING < ERROR < CRITICAL)
- Log Handlers: Multiple handlers (file, console) for different outputs
- Log Formatting: Customize log message format with timestamps, levels, etc.
- Verification Statistics: Track test results and errors for reporting

Command reference highlights

Python examples (no simulator)

- ``./scripts/module1.sh`` — runs all five Python example tracks (default)
- ``./scripts/module1.sh --decorators --async-await`` — selective example subsets
- ``python3 module1/examples/python_basics/transaction.py`` — standalone OOP/transaction drill

cocotb RTL tests

- ``./scripts/module1.sh --cocotb-tests`` — build and run cocotb regressions via orchestrator
- ``cd module1/tests/cocotb_tests && make SIM=verilator TEST=test_and_gate`` — single-test Makefile flow
- ``make SIM=verilator TEST=test_counter`` — sequential counter regression with reset/enable checks

pyuvvm entry tests

- ``./scripts/module1.sh --pyuvvm-tests`` — compile RTL and run introductory pyuvvm test against AND gate
- ``cd module1/tests/pyuvvm_tests && make SIM=verilator`` — direct Make entry when debugging pyuvvm wiring
- ``./scripts/module1.sh --all-tests`` — Python examples plus cocotb and pyuvvm test suites

Hands-on examples

Module 1 orchestrator

```
$ ./scripts/module1.sh --help
```

Terminal: module1_check

```
[[0;36m===== [[0m
[[0;36mModule 1: Python and Verification Basics [[0m
[[0;36m===== [[0m
```

Usage: ./scripts/module1.sh [OPTIONS]

Module 1: Python and Verification Basics
This script runs examples and tests for Module 1.

OPTIONS:

Python Examples:

--python-basics	Run Python class basics examples
--decorators	Run decorators and context managers examples
--async-await	Run async/await examples
--data-structures	Run data structures examples
--error-handling	Run error handling and logging examples
--all-python	Run all Python examples (default)
--skip-python	Skip all Python examples

Tests:

--cocotb-tests	Run cocotb tests
--pyuvvm-tests	Run pyuvvm tests
--all-tests	Run all tests

Environment:

--venv DIR	Virtual environment directory (default: .venv)
--no-venv	Don't use virtual environment
--sim SIMULATOR	Simulator to use (default: verilator)

Other:

--help, -h	Show this help message
------------	------------------------

EXAMPLES:

```
# Run all Python examples
./scripts/module1.sh
```

Exercise scaffold

```
./scripts/module1.sh --scaffold
```

Demo: Transaction Classes

```
$ .venv/bin/python module1/examples/python_basics/transaction.py
```

Terminal: ex01_python_basics

```
=====
Module 1 Example 1.1: Python Class Basics
=====

1. Creating base transaction:
  Transaction(id=1, data=4660, timestamp=0)
  Transaction ID: 1

2. Creating read transaction (inheritance):
  ReadTransaction(id=2, address=0x1000, data=57005)
  Address: 0x1000

3. Creating write transaction (inheritance):
  WriteTransaction(id=3, address=0x2000, data=0xBEEF)
  Address: 0x2000

4. Testing equality:
  txn1 == txn2: True
  txn1 == read_txn: False

5. Using transactions in a set (requires __hash__):
  Set size: 3
  - WriteTransaction(id=3, address=0x2000, data=0xBEEF)
  - Transaction(id=1, data=4660, timestamp=0)
  - ReadTransaction(id=2, address=0x1000, data=57005)

=====
Example completed successfully!
=====
```

Demo: Decorators and Context Managers

```
$ .venv/bin/python module1/examples/decorators/decorators_example.py
```

Terminal: ex02_decorators

```
=====
Module 1 Example 1.2: Decorators and Context Managers
=====
```

1. Using function decorators:

2. Using context manager (class-based):
Elapsed time: 0.1002s

3. Using context manager (function-based):

4. Using nested context managers:
Inner elapsed: 0.0501s
Outer elapsed: 0.0504s

```
=====
Example completed successfully!
=====
```

```
2026-05-30 20:19:37,688 - __main__ - INFO - Calling setup with args=(<__main__.VerificationTest
2026-05-30 20:19:37,689 - __main__ - INFO - Setting up test environment
2026-05-30 20:19:37,789 - __main__ - INFO - setup returned: None
2026-05-30 20:19:37,789 - __main__ - INFO - setup took 0.1005 seconds
2026-05-30 20:19:37,789 - __main__ - INFO - Running test: test_example
2026-05-30 20:19:37,839 - __main__ - INFO - run_test took 0.0502 seconds
2026-05-30 20:19:37,839 - __main__ - INFO - Tearing down test environment
2026-05-30 20:19:37,890 - __main__ - INFO - teardown took 0.0503 seconds
2026-05-30 20:19:37,890 - __main__ - INFO - Entering verification context: test_context
2026-05-30 20:19:37,990 - __main__ - INFO - Exiting verification context: test_context (success,
2026-05-30 20:19:37,990 - __main__ - INFO - Starting simulation phase: reset_phase
2026-05-30 20:19:38,041 - __main__ - INFO - Performing reset operations
2026-05-30 20:19:38,041 - __main__ - INFO - Completed simulation phase: reset_phase (0.0505s)
2026-05-30 20:19:38,041 - __main__ - INFO - Starting simulation phase: test_phase
2026-05-30 20:19:38,141 - __main__ - INFO - Running test operations
2026-05-30 20:19:38,142 - __main__ - INFO - Completed simulation phase: test_phase (0.1006s)
2026-05-30 20:19:38,142 - __main__ - INFO - Entering verification context: outer
2026-05-30 20:19:38,142 - __main__ - INFO - Entering verification context: inner
```

Demo: Async/Await Patterns

```
$ .venv/bin/python module1/examples/async_await/async_example.py
```

Terminal: ex03_async_await

```
=====
Module 1 Example 1.3: Async/Await for Simulation
=====

1. Sequential execution:
Running sequential execution example...
Final result: Step 1 complete, Step 2 complete, Step 3 complete

2. Parallel tasks:
Running parallel tasks example...
Results collected: [0, 2, 4, 6, 8]

3. Timeout handling:
Running timeout example...
Operation timed out!

4. Exception handling:
Running exception handling example...
Caught exception: Simulated error

=====
Example completed successfully!
=====
```

Demo: Data Structures

```
$ .venv/bin/python  
    module1/examples/data_structures/data_structures_example.py
```

Terminal: ex04_data_structures

```
=====
Module 1: Data Structures for Verification
=====
```

1. Transaction Queue (deque):

```
Queue size: 5
Popped: id=1, addr=0x1000, data=0x0
Popped: id=2, addr=0x1001, data=0x2
Popped: id=3, addr=0x1002, data=0x4
Popped: id=4, addr=0x1003, data=0x6
Popped: id=5, addr=0x1004, data=0x8
```

2. Scoreboard (defaultdict, Counter):

```
Matches: 5
Mismatches: 0
Unexpected: 2
```

3. Coverage Collector (set, Counter):

```
address: 60.0% coverage
  Unique values: 6
  Total hits: 7
data: 60.0% coverage
  Unique values: 6
  Total hits: 8
opcode: 50.0% coverage
  Unique values: 5
  Total hits: 5
```

4. List Comprehensions:

```
Created 10 transactions
First transaction: Transaction(id=0, address=4096, data=0, timestamp=0)
```

5. Dictionary Comprehensions:

```
Created address map with 10 entries
Sample: {4096: 0, 4097: 2, 4098: 4}
```

Demo: Error Handling and Logging

```
$ .venv/bin/python  
    module1/examples/error_handling/error_handling_example.py
```

Terminal: ex05_error_handling

```
=====
Module 1: Error Handling and Logging
=====
```

1. Basic Error Handling:

- ✓ Successful check passed
 - ✓ Caught expected error: Mismatch at address 0x2000: expected 0x5678, got 0x9ABC
- Statistics: {'total': 2, 'pass': 1, 'fail': 1, 'error': 0, 'errors': 1}

2. Exception Chaining:

- ✓ Exception chaining demonstrated

3. Retry Logic:

- ✓ Operation succeeded after 3 attempts

4. Logging Levels:

- ✓ Logging levels demonstrated (check verification.log)

```
=====
Example completed successfully!
=====
```

Check 'verification.log' for detailed logs

```
2026-05-30 20:19:40,481 - __main__ - INFO - Initialized verification checker: BasicChecker
2026-05-30 20:19:40,482 - __main__ - DEBUG - BasicChecker: Match at address 0x1000: 0x1234
2026-05-30 20:19:40,483 - __main__ - ERROR - BasicChecker: Mismatch at address 0x2000: expected
2026-05-30 20:19:40,483 - __main__ - ERROR - Caught chained exception: Verification failed
2026-05-30 20:19:40,484 - __main__ - ERROR - Original exception: Original error
2026-05-30 20:19:40,484 - __main__ - INFO - Initialized verification checker: RetryChecker
2026-05-30 20:19:40,485 - __main__ - WARNING - RetryChecker: Attempt 1 failed: Timeout in flaky_
2026-05-30 20:19:40,486 - __main__ - WARNING - RetryChecker: Attempt 2 failed: Timeout in flaky_
2026-05-30 20:19:40,486 - __main__ - INFO - RetryChecker: Operation succeeded on attempt 3
2026-05-30 20:19:40,487 - __main__ - DEBUG - This is a DEBUG message (detailed information)
2026-05-30 20:19:40,487 - __main__ - INFO - This is an INFO message (general information)
2026-05-30 20:19:40,488 - __main__ - WARNING - This is a WARNING message (potential issues)
2026-05-30 20:19:40,489 - __main__ - ERROR - This is an ERROR message (errors occurred)
2026-05-30 20:19:40,489 - __main__ - CRITICAL - This is a CRITICAL message (serious errors)
```

Practice & assessment

What you should know (1/9)

- Write Python classes with inheritance
- Use decorators and context managers effectively
- Understand and use async/await for simulation
- Explain verification fundamentals
- Understand testbench architecture
- Comprehend simulation flow

What you should know (2/9)

- Write basic assertions
- Use Python testing frameworks
- Handle errors and implement logging
- Class definition
- Instance variables
- Methods

What you should know (3/9)

- Inheritance
- Function decorators
- Async function definition
- Await usage
- Testbench structure
- Clock generation

What you should know (4/9)

- Signal driving
- Basic checking
- Test Structure: Using `@cocotb.test()` decorator
- Signal Access: Reading and writing DUT signals
(``dut.a``, ``dut.b``, ``dut.y``)
- Timing Control: Using `await Timer()` for simulation time
- Assertions: Python assertions for checking results

What you should know (5/9)

- Test Cases: Multiple test functions for different scenarios
- Truth Table Testing: Systematic testing of all input combinations
- `@cocotb.test()`: Decorator marks function as a test
- DUT Access: `dut.signal_name.value` to read/write signals
- `Timer()`: Advances simulation time
- Assertions: Python `assert` statements for checking

What you should know (6/9)

- Test Organization: Multiple test functions in one file
- Clock Generation: Creating clock signal with ``generate_clock()`` coroutine
- Reset Sequence: Implementing reset with proper timing
- Sequential Logic Testing: Testing clocked (sequential) circuits
- Enable Control: Testing enable/disable functionality
- Edge Detection: Using `` RisingEdge()`` trigger

What you should know (7/9)

- Multiple Test Scenarios: Reset, increment, enable, overflow tests
- Clock Generation: Background coroutine for clock signal
- ``cocotb.start_soon()``: Start background coroutines
- `` RisingEdge()``: Wait for clock edge
- Reset Timing: Proper reset sequence with timing
- Sequential Testing: Testing state machines and counters

What you should know (8/9)

- UVM Test Structure: ``@uvm_test()` decorator
- UVM Phases: ``build_phase()`, ``run_phase()`,
``check_phase()`
- UVM Components: ``uvm_test`, ``uvm_env`, ``uvm_agent`,
``uvm_driver`, ``uvm_monitor`
- UVM Sequences: ``uvm_sequence` and
``uvm_sequence_item`
- UVM Reporting: Using ``self.logger` for messages
- Objections: Using ``raise_objection()` and
``drop_objection()` for test control

What you should know (9/9)

- UVM Phases: Build, connect, run, check phases
- Component Hierarchy: Test → Environment → Agent → Driver/Monitor
- Sequences: Generate and send transactions
- Objections: Control test execution duration
- Factory Pattern: UVM uses factory for component creation

Exercises

- Class Design
- Async Patterns
- Testbench Structure
- Assertions
- Logging

Assessment checklist

- Can write Python classes with inheritance
- Understands decorators and context managers
- Can use async/await effectively
- Understands verification fundamentals
- Can explain testbench architecture
- Understands simulation flow

Summary & next steps

- Understand Python for verification and verification fundamentals
- Complete module1/CHECKLIST.md
- Review module1/EXAMPLES.md and run each lab
- Next: Next module in course